

Prolog Programming Assignment: Minimax Adversarial Search (Tic-Tac-Toe)

Goal. Implement basic minimax for a deterministic, two-player, zero-sum, perfect-information game.

State representation (fixed). Board is a list of 9 cells in row-major order; each cell is x, o, or e.

Required predicates.

- `move(+Board, +Player, -NextBoard)`: generate legal successor states.
- `terminal(+Board)`: true if x wins, o wins, or the board is full.
- `utility(+Board, -U)`: $U=1$ if x wins, $U=-1$ if o wins, $U=0$ if draw.
- `minimax_value(+Board, +Player, -Value)`: minimax value assuming Player to move.
- `best_move(+Board, +Player, -BestBoard, -BestValue)`: choose minimax-optimal successor.

Instrumentation (required). Use `clear_count/0`, `inc_count/0`, `get_count/1`. Increment once per evaluated board in minimax. Report expansions for at least 3 positions in your writeup.

Written questions (short).

- 1 Why is minimax optimal under the assumptions (deterministic, perfect information, zero-sum)?
- 2 What does a minimax value mean “in words”?
- 3 Why is minimax infeasible for large games like chess?

How to test. Run:

```
?- [ttt_tests].  
?- run_tests.
```

Deliverables. Submit a ZIP with `ttt_minimax.pl` and `writeup.pdf`.

Grading rubric (100 points).

- Move generator correctness: 25
- Terminal test + utility: 20
- Minimax value correctness: 30
- Best move correctness: 10
- Instrumentation + reported counts: 10
- Written questions: 5